INSTALLATION GUIDE

App version: v0.1.0-dev

This guide must be updated after installation, infrastructure, dependency, model, database, backup or environment changes.

OVERVIEW

This application runs on an Ubuntu 24.04 computer with OpenClaw, PostgreSQL, Ollama qwen3:4b, FastAPI, React and background workers.

REQUIRED SERVICES

- OpenClaw
- PostgreSQL
- Ollama with qwen3:4b
- FastAPI backend
- React frontend
- Background workers

ENVIRONMENT VARIABLES

Use .env.example as the template. Do not commit the real .env file.

POSTGRESQL SETUP

Use Docker Compose by default because the repository already includes a local-only PostgreSQL service. Native PostgreSQL is acceptable later if it is configured with the same local-only exposure and environment-variable credential rules.

PostgreSQL must not be exposed to the internet. The included Compose configuration binds PostgreSQL to localhost only:

text
127.0.0.1:5432:5432

Create the real .env from .env.example, then replace placeholder database credentials with local secrets. Do not commit .env.

bash
cp .env.example .env
EDIT POSTGRES_PASSWORD AND DATABASE_URL IN .ENV ONLY

Start PostgreSQL with Docker Compose:

bash
docker compose up -d postgres

Apply Stage 0 migrations:

bash
./scripts/db_migrate.sh
./scripts/db_status.sh

The migration runner loads .env when present, creates schema_migrations, and applies SQL files from database/migrations in filename order. The first migration enables required extensions and creates the Stage 0 model-task logging table.

Required extensions for the first migration:

- pgcrypto
- uuid-ossp

Stage 0 tables:

- schema_migrations
- model_task_logs

OLLAMA SETUP

Ollama must run locally only and the approved V1 local model is qwen3:4b.

Recommended secure defaults:

bash
export PATH="$HOME/.local/bin:$PATH"   # only needed for a user-space install
export OLLAMA_HOST=127.0.0.1:11434
ollama serve

In another terminal, install and validate the approved model:

bash
./scripts/install_ollama_qwen3_4b.sh
./scripts/test_ollama_qwen3_4b.sh

The validation scripts fail if Ollama is unreachable, if qwen3:4b is missing, or if unexpected local Ollama models are present.

PAID MODEL SETUP

Configure ChatGPT as the primary development model and Anthropic as fallback.

Stage 0 model routing policy:

1. Ollama qwen3:4b for low-risk cleanup, classification, extraction, tagging and first-pass summaries.
2. ChatGPT/OpenAI as the primary paid development/reasoning provider.
3. Anthropic as fallback when ChatGPT/OpenAI token, context, rate or quota limits block progress.
4. Latest suitable paid models only for final higher-risk production/research analysis later.

Configure API keys only in the real .env file or host secret manager:

text
OPENAI_API_KEY=...
ANTHROPIC_API_KEY=...

Do not commit real API keys. Do not expose paid provider keys to frontend code.

Fallback events are recorded through the model_task_logs foundation. Logs must not store prompts, raw responses, API keys or secrets.

BACKUP AND RESTORE

Use the backup and restore scripts in /backup.

Stage 0 scripts:

- backup/backup_postgres.sh
- backup/restore_postgres.sh
- backup/backup_app_config.sh
- backup/restore_app_config.sh

PostgreSQL backup and restore scripts load .env when present. Backups use custom-format pg_dump files and write a .sha256 checksum. Restore verifies the checksum when the checksum file is present.

Do not include unencrypted secrets in backups.

VALIDATION CHECKLIST

Run structural validation:

bash
./scripts/stage0_validate.sh

Stage 0 checklist:

- Backend skeleton imports and exposes /health
- Frontend skeleton exists and builds once Node dependencies are installed
- PostgreSQL Docker Compose config binds to localhost only
- Migration runner creates schema_migrations and Stage 0 migration creates model_task_logs
- Ollama is installed locally on 127.0.0.1:11434 and responds with qwen3:4b
- ChatGPT primary model is configured through environment/provider settings
- Anthropic fallback is configured through environment/provider settings
- Mock fallback tests cover token/rate-limit fallback behaviour
- model_task_logs migration foundation exists for fallback event logging
- ASVS register exists and records that latest stable ASVS must be checked at audit time
- No secrets are committed